MaterialApp dans Flutter

C'est quoi ?

`MaterialApp` est un **widget racine** qui enveloppe toute ton application Flutter. Tu le mets tout en haut de ton arbre de widgets, et il donne à ton app :

- Le **style Material Design** (couleurs, polices, animations, ripple effect au tap)
- La **navigation** entre écrans
- La **gestion des routes** (URLs internes)
- Les **thèmes** (clair/sombre)
- La **localisation** (langues)
- Le **titre** de l'app
- Un **debug banner** (le ruban rouge "DEBUG" en haut à droite)

Bref : sans lui, on a des widgets tout nus. Avec lui, on a une **vraie app** avec une identité visuelle cohérente.

Exemple minimal

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      title: 'Mon App',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: Text('Accueil')),
        body: Center(child: Text('Salut !')),
      ),
    ),
  );
}
```

`runApp()` prend **un seul widget** en paramètre → c'est presque toujours `MaterialApp` (ou `CupertinoApp` sur iOS, ou `WidgetsApp` si tu veux tout faire à la main).

Les paramètres qu'on va utiliser 90% du temps

Paramètre	Rôle
`home`	L'écran affiché au démarrage

Paramètre	Rôle
`initialRoute` / `routes`	Navigation par nom (`'/login'`, `'/home'`)
`onGenerateRoute`	Génération dynamique de routes (utile pour passer des arguments)
`theme`	Thème clair
`darkTheme`	Thème sombre
`themeMode`	`light`, `dark`, ou `system` (suit le téléphone)
`title`	Nom affiché dans le multitâche
`debugShowCheckedModeBanner`	`false` pour virer le ruban DEBUG
`navigatorKey`	Pour piloter la navigation depuis n'importe où
`locale` / `localizationsDelegates`	Gestion des langues
`builder`	Envelopper tout ton app (ex: MediaQuery, overlay)

Comment ça marche sous le capot

`MaterialApp` est en fait une **surcouche** au-dessus de `WidgetsApp`. Il ajoute :

1. Un `Theme` global (via `InheritedWidget`) → accessible partout avec `Theme.of(context)`
2. Les `Localizations` Material (boutons "OK", "Annuler", etc.)
3. Les transitions Material entre écrans (slide, fade…)
4. Les widgets comme `Scaffold`, `AppBar`, `FloatingActionButton` qui **dépendent** de lui pour fonctionner correctement

Si tu retires `MaterialApp`, `Scaffold` va râler : il ne trouve pas de `Material` ancestor.

Les pièges classiques

1. **Mettre `MaterialApp` plusieurs fois** → non, un seul à la racine. Pour un sous-arbre, utilise `Material` ou `Theme`.
2. **Oublier `home` ou `routes`** → écran noir, aucune erreur claire.
3. **`Navigator` avant `MaterialApp`** → erreur "No Navigator found". Le `Navigator` est **créé par** `MaterialApp`.
4. **Changer le thème à chaud** : utilise `themeMode` + un `ValueNotifier`, pas besoin de redémarrer.
5. **Confondre `MaterialApp` et `Scaffold`** : `MaterialApp` = config globale. `Scaffold` = structure d'**un** écran (AppBar + body + FAB + drawer).

Analogie simple

- `MaterialApp` = le **cadre + le règlement intérieur** de ton app (langue, couleurs, navigation)
- `Scaffold` = **une pièce** dans le bâtiment (salon, cuisine…)
- `AppBar`, `Body`, `FAB` = les **meubles** dans la pièce

Tu peux avoir plusieurs `Scaffold` (un par écran), mais un seul `MaterialApp`.

Quand NE PAS l'utiliser

- App **100% iOS-native** → `CupertinoApp`
- Jeu ou rendu custom total (Canvas, WebGL) → `WidgetsApp` seul ou `runApp` direct
- Test unitaire d'un widget → tu l'enveloppes manuellement avec `MaterialApp` **dans le test**, pas dans le code prod

Étudions les paramètres **un par un** pour qu’on l'effet de chacun.

Étape 0 — Le strict minimum

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Mon App')),
        body: Center(child: Text('Bonjour !')),
      ),
    ),
  );
}
```

Ce que nous voyons :

- `runApp(...)` → démarre l'app
- `MaterialApp(...)` → la coquille qui donne le style Material
- `home:` → l'écran affiché au démarrage
- `Scaffold` → la structure d'un écran (barre du haut + corps)
- `AppBar` → la barre du haut
- `Center` → centre son enfant
- `Text` → affiche du texte

Retiens ça par cœur. Tout le reste, ce sont des options qu'on ajoute.

Étape 1 — Enlever le ruban rouge "DEBUG"

Par défaut, Flutter affiche un ruban rouge en haut à droite. Pour le virer :

```
MaterialApp(
  debugShowCheckedModeBanner: false,   //  ajoute cette ligne
  home: Scaffold(
    appBar: AppBar(title: Text('Mon App')),
    body: Center(child: Text('Bonjour !')),
  ),
)
```

Effet : plus de ruban rouge. C'est tout.

Étape 2 — Donner un titre à l'app

```
MaterialApp(
  title: 'Ma Super App',               //  ajoute cette ligne
  debugShowCheckedModeBanner: false,
  home: Scaffold(
    appBar: AppBar(title: Text('Mon App')),
    body: Center(child: Text('Bonjour !')),
  ),
)
```

Effet : le nom affiché dans le **multitâche** du téléphone (la liste des apps ouvertes). Pas visible dans l'app elle-même.

Étape 3 — Changer la couleur

```
MaterialApp(
  title: 'Ma Super App',
  debugShowCheckedModeBanner: false,
```

```
  theme: ThemeData(                    //  ajoute ce bloc
    primarySwatch: Colors.green,
  ),
  home: Scaffold(
    appBar: AppBar(title: Text('Mon App')),
    body: Center(child: Text('Bonjour !')),
  ),
)
```

Effet : l'AppBar devient verte, les boutons aussi. Essaie `Colors.red`, `Colors.purple`, `Colors.orange` pour voir.

Étape 4 — Ajouter un 2e écran (navigation simple)

Pour l'instant, **une seule page**. On va en ajouter une deuxième, et un bouton pour y aller.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Ma Super App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.green),
      home: const PageAccueil(),
    );
  }
}

// --- Écran 1 ---
class PageAccueil extends StatelessWidget {
  const PageAccueil({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Accueil')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Aller page 2'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (_) => const PageDeux()),
            );
          },
        ),
      ),
    );
  }
}
```

```
// --- Écran 2 ---
class PageDeux extends StatelessWidget {
  const PageDeux({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Page 2')),
      body: const Center(child: Text('Me voilà !')),
    );
  }
}
```

Ce qu'il faut comprendre ici :

- `Navigator.push(...)` → empile un nouvel écran par-dessus
- `MaterialPageRoute(...)` → la "route" qui contient l'écran à afficher
- Le bouton "retour" ← apparaît automatiquement dans l'AppBar

À ce stade, **tu n'as même pas besoin de `routes:`**. `Navigator.push` suffit pour 90% des cas débutant.

Étape 5 — Navigation par nom (plus propre)

Quand on a **beaucoup d'écrans**, écrire `MaterialPageRoute` partout devient lourd. On donne un **nom** à chaque écran :

```
MaterialApp(
  title: 'Ma Super App',
  debugShowCheckedModeBanner: false,
  theme: ThemeData(primarySwatch: Colors.green),

  initialRoute: '/',                    //  écran de départ
  routes: {                             //  la table des écrans
    '/': (context) => const PageAccueil(),
    '/deux': (context) => const PageDeux(),
  },
)
```

Et dans le bouton, on remplace :

```
// AVANT
Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const PageDeux()),
);

// APRÈS
Navigator.pushNamed(context, '/deux');
```

Effet : exactement le même, mais plus court à écrire. C'est **la seule** chose qu'on doit retenir sur `routes`.

Étape 6 — Mode sombre (light/dark)

C'est le seul cas où on a besoin de transformer `MonApp` en `StatefulWidget`, parce que le thème doit **changer** quand on clique.

```
class MonApp extends StatefulWidget {
  const MonApp({super.key});
  @override
  State<MonApp> createState() => _MonAppState();
}

class _MonAppState extends State<MonApp> {
  bool _sombre = false;                  //  état : clair ou sombre

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.green),        // clair
      darkTheme: ThemeData(brightness: Brightness.dark),    // sombre
      themeMode: _sombre ? ThemeMode.dark : ThemeMode.light,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Accueil'),
          actions: [
            IconButton(
              icon: const Icon(Icons.brightness_6),
              onPressed: () {
                setState(() {
                  _sombre = !_sombre;   //  on inverse
                });
              },
            ),
          ],
        ),
        body: const Center(child: Text('Bonjour !')),
      ),
    );
  }
}
```

Les 3 lignes clés :

- `theme:` → à quoi ça ressemble en clair
- `darkTheme:` → à quoi ça ressemble en sombre
- `themeMode:` → lequel choisir **maintenant**

`setState(...)` dit à Flutter : "j'ai changé une valeur, redessine l'écran".

Récap — Ce qu'on doit vraiment retenir

Paramètre	À quoi ça sert	Obligatoire ?
`home:`	Premier écran	Oui (ou `initialRoute`)
`title:`	Nom dans le multitâche	Non
`debugShowCheckedModeBanner:`	Enlever le ruban DEBUG	Non
`theme:`	Couleurs de l'app	Non
`darkTheme:` + `themeMode:`	Mode sombre	Non
`initialRoute:` + `routes:`	Navigation par nom	Non (mais pratique)

L'ordre de la révision

1. `MaterialApp` + `home` + `Scaffold` + `AppBar` + `Text` → tu sais afficher un écran
2. `Navigator.push` + `MaterialPageRoute` → tu sais passer à un autre écran
3. `routes` + `Navigator.pushNamed` → tu sais ranger tes écrans proprement
4. `theme` + `darkTheme` + `themeMode` → tu sais changer les couleurs
5. `onGenerateRoute`, `builder`, `localizationsDelegates` → on revient **plus tard**, quand on en aura besoin

Scaffold

C'est quoi ?

`Scaffold` veut dire **échafaudage** en anglais. C'est **la structure d'UN écran**.

Si `MaterialApp` c'est le **bâtiment entier** (avec ses règles, ses couleurs, sa langue), alors `Scaffold` c'est **une pièce** : le salon, la cuisine, la chambre…

Une app = plusieurs écrans = plusieurs `Scaffold`.

Chaque `Scaffold` te donne **gratuitement** les éléments standards d'un écran mobile :

- Une barre en haut (AppBar)
- Une zone de contenu au milieu (body)

- Une barre en bas (BottomNavigationBar)
- Un bouton flottant (FloatingActionButton)
- Un menu latéral qui glisse (Drawer)
- Un panneau qui glisse depuis la droite (endDrawer)
- Un espace pour le clavier (resizeToAvoidBottomInset)
- etc.

Tu n'as **rien à positionner à la main** : tu remplis les cases, Flutter les place au bon endroit.

Étape 0 — Le strict minimum

```
Scaffold(
  body: Center(child: Text('Bonjour !')),
)
```

Ce qu'on a : juste du texte au milieu. Pas de barre, pas de bouton. C'est tout.

Un `Scaffold` peut être **vide** comme ça. Le reste, ce sont des options que tu ajoutes **quand tu en as besoin**.

Important : `Scaffold` doit toujours être **à l'intérieur** d'un `MaterialApp` (sinon erreur "No Material widget found").

Étape 1 — Ajouter une barre en haut (AppBar)

```
Scaffold(
  appBar: AppBar(                      //  ajoute cette ligne
    title: Text('Mon écran'),
  ),
  body: Center(child: Text('Bonjour !')),
)
```

Effet : une barre colorée en haut avec le titre. Le corps passe automatiquement **en dessous**.

Étape 2 — Ajouter un bouton flottant

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  body: Center(child: Text('Bonjour !')),
  floatingActionButton: FloatingActionButton(   //  ajoute ce bloc
    onPressed: () {
      print('Cliqué !');
```

```
    },
    child: Icon(Icons.add),
  ),
)
```

Effet : un bouton rond en bas à droite, qui flotte au-dessus du contenu. Le classique bouton "+" des apps.

Étape 3 — Ajouter un menu latéral (Drawer)

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  drawer: Drawer(                              //  ajoute ce bloc
    child: ListView(
      children: [
        DrawerHeader(child: Text('Menu')),
        ListTile(title: Text('Accueil')),
        ListTile(title: Text('Paramètres')),
      ],
    ),
  ),
  body: Center(child: Text('Bonjour !')),
)
```

Effet : une **icône ≡** apparaît automatiquement à gauche de l'AppBar. En cliquant dessus, un panneau glisse depuis la gauche.

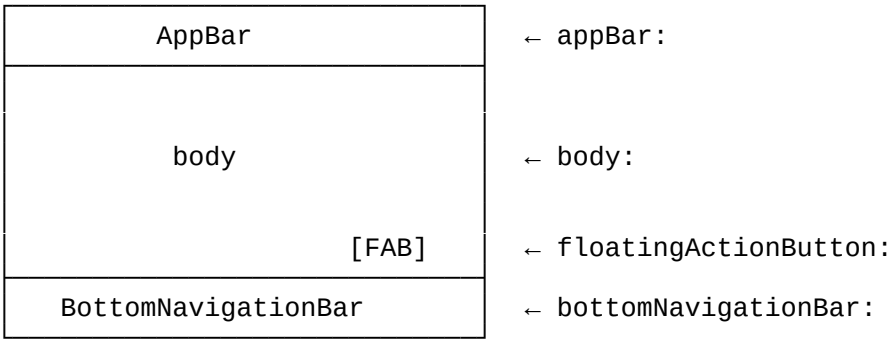
Tu n'as **rien à coder** pour le bouton **≡** → `Scaffold` le crée tout seul dès que tu fournis un `drawer`.

Étape 4 — Ajouter une barre en bas

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  body: Center(child: Text('Bonjour !')),
  bottomNavigationBar: BottomNavigationBar(    //  ajoute ce bloc
    items: [
      BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Accueil'),
      BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profil'),
    ],
  ),
)
```

Effet : une barre fixe tout en bas avec des onglets. Le corps se **rétrécit** automatiquement pour laisser la place.

Récap visuel : les zones d'un Scaffold



Tous les paramètres importants

Paramètre	Rôle	Exemple
`appBar`	Barre en haut	`AppBar(title: Text('...'))`
`body`	Contenu principal	N'importe quel widget
`floatingActionButton`	Bouton rond flottant	`FloatingActionButton(...)`
`floatingActionButtonLocation`	Où le placer	`FloatingActionButtonLocation.centerDock`
`drawer`	Menu qui glisse depuis la gauche	`Drawer(child: ...)`
`endDrawer`	Menu qui glisse depuis la droite	`Drawer(child: ...)`
`bottomNavigationBar`	Barre en bas	`BottomNavigationBar(...)`

Paramètre	Rôle	Exemple
`bottomSheet`	Feuille qui reste affichée en bas	`Container(...)`
`backgroundColor`	Couleur de fond	`Colors.grey[200]`
`resizeToAvoidBottomInset`	Le body remonte quand le clavier apparaît	`true` (défaut) / `false`
`extendBody`	Le body passe **derrière** la bottom bar	`true` / `false`
`extendBodyBehindAppBar`	Le body passe **derrière** l'AppBar	`true` / `false`
`persistentFooterButtons`	Boutons fixes juste au-dessus du bas	`[ElevatedButton(...)]`

Les 3 pièges classiques du débutant

Piège 1 — Oublier `MaterialApp` au-dessus

```
//  ERREUR : No Material widget found
runApp(Scaffold(body: Text('Salut')));
```

`Scaffold` a besoin d'un `MaterialApp` **au-dessus** de lui pour fonctionner.

Piège 2 — Le clavier cache le contenu

Par défaut, quand tu cliques dans un `TextField` en bas de l'écran, le clavier cache le champ.

Solution simple : mets ton contenu dans un `SingleChildScrollView` :

```
Scaffold(
  body: SingleChildScrollView(       //  enveloppe
    child: Column(
      children: [
        TextField(),
        // ... beaucoup de champs
      ],
    ),
```

```
  ),
)
```

Le `Scaffold` remonte automatiquement le body quand le clavier apparaît (`resizeToAvoidBottomInset: true` par défaut).

Piège 3 — Confondre `Scaffold` et `MaterialApp`

	`MaterialApp`	`Scaffold`
Combien par app ?	**1 seul**	**1 par écran**
Rôle	Config globale (thème, langue, navigation)	Structure d'un écran
Contient	Des routes, des thèmes	Une AppBar, un body…
Se met où ?	À la racine (`runApp`)	Dans chaque page

Analogie pour bien mémoriser

- `MaterialApp` = **le bâtiment** avec ses règles (couleurs, langue, plan des étages)
- `Scaffold` = **une pièce** du bâtiment
- `AppBar` = **le panneau au-dessus de la porte** de la pièce
- `body` = **le sol** de la pièce
- `drawer` = **une porte secrète** sur le côté
- `bottomNavigationBar` = **le tapis** au fond de la pièce
- `FloatingActionButton` = **un meuble flottant** au-dessus du sol

Exemple complet — tous les paramètres ensemble

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const MonEcran(),
    );
```

```
  }
}

class MonEcran extends StatelessWidget {
  const MonEcran({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(

      //  Barre du haut
      appBar: AppBar(
        title: const Text('Mon écran'),
      ),

      //  Menu latéral gauche (icône ≡ automatique)
      drawer: Drawer(
        child: ListView(
          children: const [
            DrawerHeader(child: Center(child: Text('Menu'))),
            ListTile(leading: Icon(Icons.home), title: Text('Accueil')),
            ListTile(leading: Icon(Icons.settings), title: Text('Réglages')),
          ],
        ),
      ),

      //  Contenu principal
      body: const Center(
        child: Text('Bonjour !', style: TextStyle(fontSize: 24)),
      ),

      //  Bouton rond flottant
      floatingActionButton: FloatingActionButton(
        onPressed: () => print('Ajout !'),
        child: const Icon(Icons.add),
      ),

      // Barre en bas
      bottomNavigationBar: BottomNavigationBar(
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Accueil'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profil'),
        ],
      ),
    );
  }
}
```

Ce qu'on doit retenir (version courte)

1. `Scaffold` = **structure d'un écran**
2. Il a **5 zones principales** : appBar, body, drawer, FAB, bottomNavigationBar
3. Tu **remplis seulement les zones dont tu as besoin** → les autres disparaissent

4. Il **doit** être dans un `MaterialApp`
5. Le clavier : entoure ton body de `SingleChildScrollView` si tu as des champs en bas

AppBar

C'est quoi ?

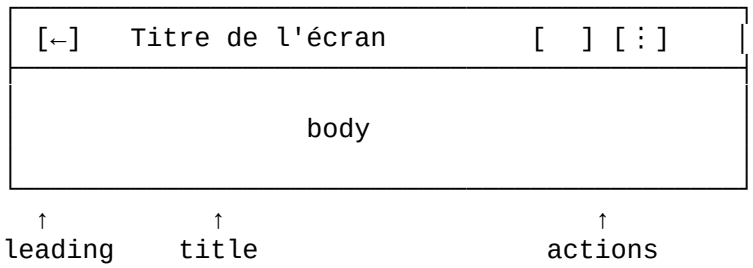
`AppBar` = **la barre en haut d'un écran**. Celle où il y a le titre, la flèche retour, parfois des icônes à droite.

Tu la mets dans le paramètre `appBar:` d'un `Scaffold` :

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  body: ...,
)
```

`AppBar` ne marche **pas** tout seul. Il a besoin d'être **dans un Scaffold** (sinon erreur).

À quoi ça ressemble (schéma)



Trois zones principales :

- **leading** (gauche) → flèche retour, ou icône ≡
- **title** (centre) → le texte principal
- **actions** (droite) → liste d'icônes/boutons

Étape 0 — Le minimum

```
AppBar(title: Text('Mon écran'))
```

Effet : une barre colorée en haut avec un texte centré (ou à gauche selon la plateforme).

Étape 1 — Ajouter des icônes à droite (`actions`)

```
AppBar(
  title: Text('Mon écran'),
  actions: [                          //  liste d'icônes à droite
    IconButton(
      icon: Icon(Icons.search),
      onPressed: () => print('Recherche'),
    ),
    IconButton(
      icon: Icon(Icons.more_vert),
      onPressed: () => print('Menu'),
    ),
  ],
)
```

Effet : deux icônes apparaissent à droite du titre. **L'ordre** de la liste = l'ordre de gauche à droite.

Étape 2 — Remplacer le bouton retour (`leading`)

Par défaut, quand tu navigues vers un nouvel écran, Flutter met **automatiquement** une flèche ← à gauche.

Pour la **remplacer** par autre chose (ex : un ≡ pour ouvrir un drawer) :

```
AppBar(
  leading: IconButton(                //  remplace la flèche par défaut
    icon: Icon(Icons.menu),
    onPressed: () => print('Menu'),
  ),
  title: Text('Mon écran'),
)
```

Attention : si tu définis `leading` **ET** un `drawer` dans le Scaffold, c'est ton `leading` qui gagne. Le ≡ auto **n'apparaît plus**.

Étape 3 — Mettre une barre de recherche / onglets en dessous (`bottom`)

```
AppBar(
  title: Text('Mon écran'),
  bottom: TabBar(                    //  un widget SOUS le titre
    tabs: [
      Tab(text: 'Accueil'),
      Tab(text: 'Profil'),
    ],
  ),
)
```

Effet : l'AppBar devient **plus haute** et affiche des onglets juste en dessous du titre.

Tu peux mettre n'importe quel widget dans `bottom:` (un `TextField` pour une barre de recherche, un `PreferredSize` custom, etc.).

Étape 4 — Changer la couleur

```
AppBar(
  title: Text('Mon écran'),
  backgroundColor: Colors.green,      //  couleur de fond
  foregroundColor: Colors.white,      //  couleur du texte et des icônes
)
```

Effet : AppBar verte avec texte blanc.

`foregroundColor` change **tout d'un coup** : titre, icônes, flèche retour. Beaucoup plus simple que de colorier chaque élément.

Étape 5 — Enlever l'ombre (`elevation`)

```
AppBar(
  title: Text('Mon écran'),
  elevation: 0,                       //  pas d'ombre sous la barre
)
```

Effet : la barre se fond dans le body, sans ligne de séparation. Très courant dans le design moderne.

Par défaut, `elevation` est à 4 (une légère ombre apparaît quand tu scrolles le contenu).

Étape 6 — Centrer ou non le titre

```
AppBar(
  title: Text('Mon écran'),
  centerTitle: true,                  //  titre centré (défaut iOS)
)
```

- `true` → titre au centre (style iPhone)
- `false` → titre à gauche (style Android, par défaut)

Tous les paramètres importants

Paramètre	Rôle	Exemple
`title`	Texte/Widget principal	`Text('Accueil')`
`leading`	Widget à gauche (remplace la flèche)	`IconButton(...)`
`actions`	Liste de widgets à droite	`[IconButton(...), IconButton(...)]`
`bottom`	Widget SOUS le titre	`TabBar(...)`, `PreferredSize(...)`
`backgroundColor`	Couleur de fond	`Colors.green`
`foregroundColor`	Couleur texte + icônes	`Colors.white`
`elevation`	Ombre sous la barre	`0`, `4`, `8`
`centerTitle`	Titre centré ?	`true` / `false`
`automaticallyImplyLeading`	Mettre la flèche retour auto ?	`true` (défaut) / `false`
`toolbarHeight`	Hauteur de la barre	`56` (défaut), `80`
`shape`	Forme custom (bord arrondi, etc.)	`RoundedRectangleBorder(...)`
`flexibleSpace`	Couche d'arrière-plan (gradient, image)	`FlexibleSpaceBar(...)`

Les pièges classiques du débutant

Piège 1 — `AppBar` sans `Scaffold` → erreur

```
// ERREUR : No Material widget found
runApp(AppBar(title: Text('Salut')));
```

`AppBar` **doit** être dans le paramètre `appBar:` d'un `Scaffold`.

Piège 2 — Vouloir cacher la flèche retour

Par défaut, quand tu fais `Navigator.push(...)` vers un nouvel écran, la flèche ← apparaît **toute seule**. Pour la supprimer :

```
AppBar(
  title: Text('Mon écran'),
  automaticallyImplyLeading: false,    //  enlève la flèche auto
)
```

Piège 3 — Mettre un `drawer` ET un `leading` en même temps

```
//  Le ≡ automatique n'apparaîtra PAS si tu définis leading
Scaffold(
  appBar: AppBar(
    leading: IconButton(icon: Icon(Icons.search), onPressed: ...),
  ),
  drawer: Drawer(...),   // le ≡ auto est "écrasé" par leading
)
```

Si tu veux **garder** le ≡ automatique du drawer, **ne touche pas** à `leading`.

Piège 4 — Titre trop long qui déborde

Un `Text` long dans `title:` peut provoquer un débordement visuel. Solution :

```
AppBar(
  title: Text(
    'Un titre vraiment très très long',
    overflow: TextOverflow.ellipsis,   //  coupe avec "..."
  ),
)
```

Analogie pour mémoriser

`AppBar` = **le panneau d'entrée d'une pièce** :

- `leading` → la **poignée de porte** à gauche
- `title` → l'**étiquette** au milieu
- `actions` → les **boutons/interrupteurs** à droite
- `bottom` → une **affiche collée sous le panneau**
- `backgroundColor` / `foregroundColor` → la **peinture** du panneau

Exemple complet — tous les paramètres ensemble

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.deepPurple),
      home: const MonEcran(),
    );
  }
}

class MonEcran extends StatelessWidget {
  const MonEcran({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      //  AppBar avec plein d'options
      appBar: AppBar(
        //  Titre
        title: const Text('Mon AppBar'),

        //  Titre centré
        centerTitle: true,

        //  Icône à gauche
        leading: IconButton(
          icon: const Icon(Icons.arrow_back),
          onPressed: () => print('Retour'),
        ),
```

```
        //  Icônes à droite
        actions: [
          IconButton(
            icon: const Icon(Icons.search),
            onPressed: () => print('Recherche'),
          ),
          IconButton(
            icon: const Icon(Icons.more_vert),
            onPressed: () => print('Menu'),
          ),
        ],

        //  Couleurs
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,

        //  Pas d'ombre
        elevation: 0,

        //  Onglets en dessous du titre
        bottom: const TabBar(
          indicatorColor: Colors.white,
          labelColor: Colors.white,
          unselectedLabelColor: Colors.white70,
          tabs: [
            Tab(text: 'Accueil'),
            Tab(text: 'Profil'),
            Tab(text: 'Réglages'),
          ],
        ),
      ),

      body: const Center(child: Text('Contenu de l\'écran')),
    );
  }
}
```

Resultat : AppBar violette avec :

- une flèche retour à gauche
- un titre centré
- deux icônes à droite
- trois onglets en dessous

Ce qu'on doit retenir (version courte)

1. `AppBar` = **barre du haut d'un écran**
2. **3 zones principales** : `leading` (gauche), `title` (centre), `actions` (droite)
3. `bottom:` = widget **sous** le titre (souvent un `TabBar`)
4. `foregroundColor` = **tout colorier d'un coup** (texte + icônes)

5. La flèche retour apparaît **toute seule** → mets `automaticallyImplyLeading: false` pour la virer
6. Elle doit être **dans un Scaffold**

Container

C'est quoi ?

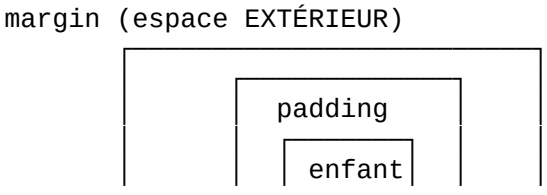
`Container` = **une boîte**. C'est tout.

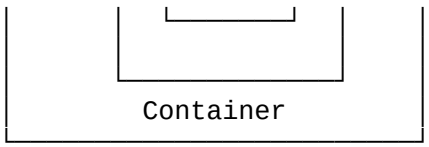
Une boîte dans laquelle tu peux :

- Mettre quelque chose dedans (un texte, une icône, un bouton…)
- Donner une **taille** (largeur, hauteur)
- Donner une **couleur** de fond
- Ajouter des **bords arrondis**
- Ajouter une **bordure**
- Ajouter de l'**espace intérieur** (padding)
- Ajouter de l'**espace extérieur** (margin)
- Mettre une **ombre** derrière

C'est **le widget le plus utilisé de Flutter** après `Text`. Tu vas en mettre partout.

Image mentale





- **margin** = la marge **autour** du Container (espace avec les voisins)
- **padding** = l'espace **entre le bord** du Container et son enfant
- **child** = ce qu'il y a à l'intérieur

Étape 0 — Le Container vide

```
Container()
```

Ce qu'on a : rien. Littéralement invisible. Un Container **sans rien dedans** ne prend aucune place (taille 0×0).

Un `Container` tout seul ne sert **à rien**. Il faut lui donner **au moins une chose** : une taille, une couleur, ou un enfant.

Étape 1 — Une boîte colorée avec une taille

```
Container(
  width: 200,                  //  largeur
  height: 100,                 //  hauteur
  color: Colors.blue,          //  couleur de fond
)
```

Effet : un rectangle bleu de 200 pixels de large et 100 de haut.

Règle importante : `width` et `height` sont en **pixels logiques** (pas en cm). Sur un écran de téléphone, 200×100 c'est petit mais visible.

Étape 2 — Mettre quelque chose dedans (`child`)

```
Container(
  width: 200,
  height: 100,
  color: Colors.blue,
  child: Text('Salut'),        //  l'enfant
)
```

Effet : le texte "Salut" apparaît dans le coin supérieur gauche de la boîte bleue.

Pourquoi en haut à gauche ? Parce qu'un `Container` **aligne son enfant en haut à gauche par défaut**.

Étape 3 — Centrer l'enfant (`alignment`)

```
Container(
  width: 200,
  height: 100,
  color: Colors.blue,
  alignment: Alignment.center,  //  centre l'enfant
  child: Text('Salut'),
)
```

Effet : le texte est maintenant au **milieu** de la boîte.

Autres valeurs utiles :

- `Alignment.topLeft` (défaut)
- `Alignment.topCenter`
- `Alignment.topRight`
- `Alignment.centerLeft`
- `Alignment.center`
- `Alignment.centerRight`
- `Alignment.bottomLeft`
- `Alignment.bottomCenter`
- `Alignment.bottomRight`

Tu peux aussi utiliser des valeurs **précises** entre -1 et 1 :

```
alignment: Alignment(0, -0.5),   // x=0 (centre), y=-0.5 (haut)
```

Étape 4 — Espace autour de l'enfant (`padding`)

Le texte colle aux bords de la boîte. On ajoute de l'espace intérieur :

```
Container(
  width: 200,
  height: 100,
  color: Colors.blue,
  alignment: Alignment.center,
  padding: EdgeInsets.all(20),   //  20px d'espace tout autour
  child: Text('Salut'),
)
```

Effet : le texte a maintenant 20px d'air autour de lui, à l'intérieur de la boîte bleue.

Les variantes de `EdgeInsets` :

```
padding: EdgeInsets.all(20),                          // 20 partout
padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10),  // H et V séparés
padding: EdgeInsets.only(left: 10, top: 5),           // côtés précis
padding: EdgeInsets.fromLTRB(10, 5, 20, 15),          // Left, Top, Right,
Bottom
```

Étape 5 — Espace autour de la boîte (`margin`)

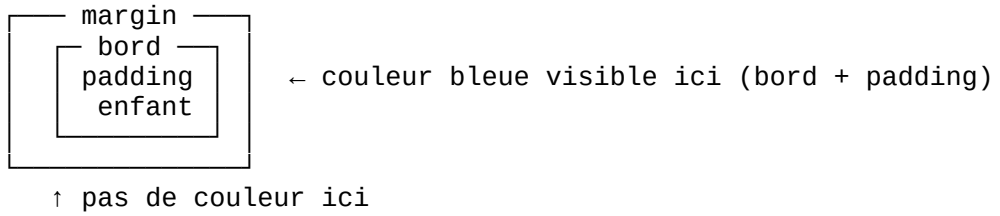
Pour éloigner la boîte de ses voisins :

```
Container(
  width: 200,
  height: 100,
  color: Colors.blue,
  margin: EdgeInsets.all(30),    //  30px AUTOUR de la boîte
  child: Text('Salut'),
)
```

Effet : la boîte bleue est poussée de 30px de tous les côtés par rapport à son parent.

Différence padding vs margin — retiens bien :

- `padding` → espace **dedans**, la couleur s'étend jusqu'au padding
- `margin` → espace **dehors**, la couleur s'arrête avant



Étape 6 — Bords arrondis et bordure

```
Container(
  width: 200,
  height: 100,
  decoration: BoxDecoration(           //  remplace "color:"
    color: Colors.blue,
    borderRadius: BorderRadius.circular(16),   //  coins arrondis
    border: Border.all(                        //  bordure
      color: Colors.black,
      width: 2,
    ),
  ),
  child: Text('Salut'),
)
```

Règle cruciale : dès que tu utilises `decoration:`, tu **ne peux plus** utiliser `color:`. La couleur doit aller **dans** `BoxDecoration`.

```
//  ERREUR
Container(
  color: Colors.blue,
  decoration: BoxDecoration(borderRadius: ...),   // conflit !
)

//  CORRECT
Container(
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(16),
  ),
)
```

Étape 7 — Une ombre derrière (`boxShadow`)

```
Container(
  width: 200,
  height: 100,
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(16),
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.2),   // noir à 20% d'opacité
        blurRadius: 10,                          // flou de l'ombre
        offset: Offset(0, 4),                    // décalage (x, y)
      ),
    ],
  ),
  child: Text('Salut'),
)
```

Effet : une carte blanche avec coins arrondis et ombre en dessous. **C'est le style "card" classique** que tu vois partout.

Étape 8 — Un dégradé (gradient) au lieu d'une couleur

```
Container(
  width: 200,
  height: 100,
  decoration: BoxDecoration(
    gradient: LinearGradient(                    //  dégradé
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
      colors: [Colors.purple, Colors.blue],
    ),
  ),
  child: Text('Salut'),
)
```

Effet : un dégradé du violet (en haut à gauche) au bleu (en bas à droite).

Tous les paramètres importants

Paramètre	Rôle	Exemple
`child`	Ce qu'il y a dedans	`Text('Salut')`
`width`	Largeur fixe	`200`
`height`	Hauteur fixe	`100`
`color`	Couleur de fond (si pas de `decoration`)	`Colors.blue`
`decoration`	Décorations avancées (remplace `color`)	`BoxDecoration(...)`
`padding`	Espace INTÉRIEUR	`EdgeInsets.all(20)`
`margin`	Espace EXTÉRIEUR	`EdgeInsets.all(30)`
`alignment`	Position de l'enfant	`Alignment.center`
`constraints`	Contraintes min/max de taille	`BoxConstraints(maxWidth: 300)`
`transform`	Rotation, zoom, déplacement	`Matrix4.rotationZ(0.1)`

Les pièges classiques

Piège 1 — `color` + `decoration` en même temps

```
//  ERREUR : Cannot provide both a color and a decoration
Container(
  color: Colors.blue,
  decoration: BoxDecoration(borderRadius: BorderRadius.circular(8)),
)
```

Solution : mets la couleur **dans** `decoration` :

```
Container(
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(8),
  ),
)
```

Piège 2 — Container sans taille dans une Row/Column

```
//  Le Container prend TOUT l'espace disponible
Column(
  children: [
    Container(color: Colors.blue),      // prend toute la hauteur
    Text('Salut'),
```

```
  ],
)
```

Pourquoi ? Un `Container` sans `width`/`height` **s'étire** pour remplir l'espace que son parent lui donne.

Solution : donne-lui une taille, ou mets-le dans un `SizedBox`/`Expanded`.

Piège 3 — Container avec child ET width/height trop petits

```
//  Le texte déborde
Container(
  width: 50,
  height: 20,
  child: Text('Un texte très long'),
)
```

Un `Container` avec taille fixe **ne s'adapte pas** au contenu. Le texte est coupé ou fait une erreur "overflow".

Solution : soit tu agrandis, soit tu enlèves `width`/`height` (il s'adaptera au contenu).

Piège 4 — Confondre `padding` et `margin`

```
Container(color: Colors.blue, padding: EdgeInsets.all(20))
// → la couleur bleue s'étend JUSQU'AUX bords, le texte a 20px d'air DEDANS

Container(color: Colors.blue, margin: EdgeInsets.all(20))
// → la couleur bleue est RÉTRÉCIE, il y a 20px d'air AUTOUR (invisible)
```

Test simple : si tu mets `padding`, la **boîte colorée reste grande**. Si tu mets `margin`, la **boîte colorée rétrécit**.

Exemple complet — tous ensemble

```
import 'package:flutter/material.dart';

void main() => runApp(const MonApp());

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        backgroundColor: Colors.grey[200],
        appBar: AppBar(title: const Text('Container')),
        body: Center(
          child: Container(
            //  Taille
            width: 250,
            height: 150,

            //  Espace AUTOUR de la boîte
            margin: const EdgeInsets.all(20),

            //  Espace INTÉRIEUR
            padding: const EdgeInsets.all(20),

            //  Décorations
            decoration: BoxDecoration(
              // Couleur de fond
              color: Colors.white,

              // Coins arrondis
              borderRadius: BorderRadius.circular(16),

              // Bordure
              border: Border.all(color: Colors.deepPurple, width: 2),

              // Ombre
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.2),
                  blurRadius: 12,
                  offset: const Offset(0, 6),
                ),
              ],
            ),

            //  Position de l'enfant
            alignment: Alignment.center,

            // Contenu
            child: const Text(
              'Salut ',
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
                color: Colors.deepPurple,
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Row et Column

C'est quoi ?

- **`Row`** = aligner des widgets **horizontalement** (côte à côte →)
- **`Column`** = aligner des widgets **verticalement** (empilés ↓)

C'est **la base de tout layout** en Flutter. Dès que tu as plusieurs widgets à placer, tu utilises l'un des deux (ou les deux imbriqués).

Image simple :

```
Row →    [A] [B] [C]

Column → [A]
         [B]
         [C]
```

Un `Row` ou `Column` prend une **liste** de widgets appelée **`children`**.

Étape 0 — Le minimum

Row

```
Row(
  children: [
    Text('A'),
    Text('B'),
    Text('C'),
  ],
)
```

Effet : `A B C` côte à côte.

Column

```
Column(
  children: [
    Text('A'),
    Text('B'),
    Text('C'),
  ],
)
```

Effet : `A`, `B`, `C` empilés verticalement.

Règle d'or : `Row` et `Column` ont besoin d'un **parent qui leur donne une taille**. Si tu les mets tout seuls dans un `Scaffold.body:`, ça marche. Si tu les mets dans un autre `Row`/`Column` sans contrainte → erreur.

Étape 1 — Aligner sur l'axe principal (`mainAxisAlignment`)

C'est **l'alignement le long de la direction** :

- Pour un `Row` → **horizontal** (gauche ↔ droite)
- Pour une `Column` → **vertical** (haut ↕ bas)

```
Row(
  mainAxisAlignment: MainAxisAlignment.center,   //  centre horizontalement
  children: [Text('A'), Text('B'), Text('C')],
)
```

Les valeurs possibles :

Valeur	Effet (Row)	Effet (Column)
`start` (défaut)	Collé à gauche	Collé en haut
`center`	Centré	Centré
`end`	Collé à droite	Collé en bas
`spaceBetween`	Espace **entre** les enfants	idem
`spaceAround`	Espace **autour** de chaque	idem
`spaceEvenly`	Espaces **égaux** partout	idem

Visualisation rapide (Row avec 3 enfants) :

```
start:        [A][B][C]..........
center:       .....[A][B][C]......
end:          ..........[A][B][C]
spaceBetween: [A]....[B]....[C]
spaceAround:  .[A]...[B]...[C].
spaceEvenly:  ..[A]..[B]..[C]..
```

Étape 2 — Aligner sur l'axe secondaire (`crossAxisAlignment`)

C'est **l'alignement perpendiculaire** :

- Pour un `Row` → **vertical** (haut ↕ bas)

- Pour une `Column` → **horizontal** (gauche ↔ droite)

```
Row(
  crossAxisAlignment: CrossAxisAlignment.start,   //  alignés en HAUT
  children: [Text('A'), Text('B'), Text('C')],
)
```

Les valeurs possibles :

Valeur	Effet (Row)	Effet (Column)
`start`	Alignés en **haut**	Alignés à **gauche**
`center` (défaut)	Alignés au **milieu**	Alignés au **milieu**
`end`	Alignés en **bas**	Alignés à **droite**
`stretch`	Étirés sur toute la hauteur	Étirés sur toute la largeur
`baseline`	Alignés sur la ligne de base du texte	(rare en Column)

Visualisation (Row avec 3 enfants de hauteurs différentes) :

```
start:        end:          center:       stretch:
[A]           [ ]           [ ]           [AAA]
[B]           [ ]           [B]           [BBB]
[C]           [A]           [ ]           [CCC]
              [B]           [A]
              [C]           [B]
                            [C]
```

`stretch` est **très pratique** : tous les enfants prennent la même hauteur (Row) ou largeur (Column).

Étape 3 — L'espace entre les enfants (`spacing`)

Depuis Flutter 3.27, `Row` et `Column` ont un paramètre **`spacing`** :

```
Row(
  spacing: 20,                          //  20px entre chaque enfant
  children: [Text('A'), Text('B'), Text('C')],
)
```

Effet : `A  B  C` avec 20px d'écart entre chaque.

Avant cette version, on utilisait des `SizedBox(width: 20)` entre chaque enfant. Si tu vois ce pattern dans du vieux code, c'est normal :

```
// Ancienne méthode (toujours valide)
Row(
  children: [
    Text('A'),
    SizedBox(width: 20),    // espace
    Text('B'),
    SizedBox(width: 20),
    Text('C'),
  ],
)
```

Pour Column, c'est `SizedBox(height: 20)`.

Étape 4 — Partager l'espace (`Expanded`)

Souvent, tu veux qu'un enfant **prenne tout l'espace restant**. On l'entoure d'`Expanded` :

```
Row(
  children: [
    Text('Gauche'),
    Expanded(                          //  prend TOUT l'espace restant
      child: Container(color: Colors.blue),
    ),
    Text('Droite'),
  ],
)
```

Effet : "Gauche" et "Droite" gardent leur taille, et le Container bleu **remplit tout le milieu**.

Partager en proportions avec `flex` :

```
Row(
  children: [
    Expanded(flex: 1, child: Container(color: Colors.red)),    // 1/3
    Expanded(flex: 2, child: Container(color: Colors.blue)),   // 2/3
  ],
)
```

Effet : rouge prend 1/3 de la largeur, bleu 2/3.

Étape 5 — `Flexible` (variante souple d'`Expanded`)

`Expanded` = "prends TOUT".
`Flexible` = "prends ce dont tu as besoin, mais pas plus que l'espace dispo".

```
Row(
  children: [
    Flexible(                          //  prend ce qu'il peut, sans forcer
      child: Text('Un texte très long qui pourrait déborder...'),
    ),
    Icon(Icons.star),
  ],
)
```

Différence concrète :

- `Expanded` **force** l'enfant à remplir tout l'espace
- `Flexible` **laisse** l'enfant choisir sa taille (mais limite le débordement)

En pratique : utilise `Expanded` quand tu veux remplir, `Flexible` quand tu veux juste éviter le débordement.

Étape 6 — Row/Column imbriqués

C'est là que ça devient puissant. Tu peux mettre des `Row` dans des `Column` et inversement.

```
Column(
  children: [
    // Ligne 1 : deux boutons côte à côte
    Row(
      children: [
        Expanded(child: ElevatedButton(onPressed: () {}, child: Text('OK'))),
        SizedBox(width: 10),
        Expanded(child: ElevatedButton(onPressed: () {}, child:
Text('Annuler'))),
      ],
    ),

    SizedBox(height: 20),

    // Ligne 2 : une icône + du texte
    Row(
      children: [
        Icon(Icons.info),
        SizedBox(width: 8),
        Text('Information'),
      ],
    ),
  ],
)
```

Effet : une colonne avec deux lignes différentes. **C'est exactement comme ça qu'on construit n'importe quel écran.**

Étape 7 — Faire scroller une Column trop longue

Une `Column` **ne scrolle pas** par défaut. Si elle dépasse l'écran → erreur rouge "overflow".

Solution : enveloppe-la dans un `SingleChildScrollView` :

```
SingleChildScrollView(                //  rend la Column scrollable
  child: Column(
    children: [
      // ... 50 widgets
    ],
  ),
)
```

Ne mets JAMAIS `Expanded` dans une Column à l'intérieur d'un `SingleChildScrollView` → erreur (l'espace est infini, `Expanded` ne sait pas quoi prendre).

Tous les paramètres importants (Row et Column)

Paramètre	Rôle	Exemple
`children`	Liste des widgets (OBLIGATOIRE)	`[Text('A'), Text('B')]`
`mainAxisAlignment`	Alignement sur l'axe principal	`MainAxisAlignment.center`
`crossAxisAlignment`	Alignement sur l'axe secondaire	`CrossAxisAlignment.start`
`mainAxisSize`	Prend toute la place ou juste le nécessaire	`MainAxisSize.min` / `.max`
`spacing`	Espace entre les enfants (Flutter 3.27+)	`20`
`textDirection`	Sens (gauche→droite ou inverse)	`TextDirection.ltr`
`verticalDirection`	Ordre vertical (Column)	`VerticalDirection.down`

`mainAxisSize` — à quoi ça sert ?

Par défaut, un `Row` prend **toute la largeur** disponible, une `Column` prend **toute la hauteur** disponible.

```
Row(
  mainAxisSize: MainAxisSize.min,   //  prend juste la taille du contenu
  children: [Text('A'), Text('B')],
)
```

Quand c'est utile : quand tu mets un `Row` dans un `Row` ou un `Column` dans un `Column` → sinon ils s'étirent.

Les pièges classiques

Piège 1 — Overflow (le fameux ruban jaune/noir)

```
//  ERREUR : "A RenderFlex overflowed by 42 pixels on the right"
Row(
  children: [
    Icon(Icons.star),
    Text('Un texte vraiment très long qui dépasse la largeur de l'écran...'),
  ],
)
```

Solutions possibles :

1. Entoure le `Text` d'`Expanded` ou `Flexible`
2. Entoure-le d'un `SingleChildScrollView`
3. Réduis la taille du texte

```
//  OK
Row(
```

```
  children: [
    Icon(Icons.star),
    Expanded(                              //  le texte s'adapte
      child: Text('Un texte vraiment très long...'),
    ),
  ],
)
```

Piège 2 — `Expanded` en dehors d'un `Row`/`Column`

```
//  ERREUR : Expanded ne peut être qu'enfant de Row/Column/Flex
Container(
  child: Expanded(child: Text('Salut')),
)
```

`Expanded` **ne marche que** dans un `Row`, `Column` ou `Flex`. Point.

Piège 3 — `Row`/`Column` sans taille dans un parent

```
//  ERREUR : les contraintes sont infinies
Column(
  children: [
    Column(                                // Column dans Column = problème
      children: [Text('A'), Text('B')],
    ),
  ],
)
```

Pourquoi ? La `Column` extérieure dit "prends toute la hauteur", mais la `Column` intérieure ne sait pas **quelle** hauteur prendre.

Solution :

```
Column(
  children: [
    Column(
      mainAxisSize: MainAxisSize.min,      //  prend juste ce qu'il faut
      children: [Text('A'), Text('B')],
    ),
  ],
)
```

Piège 4 — Confondre mainAxisAlignment et crossAxisAlignment

Astuce mnémotechnique :

- **main** = la **direction** du layout (row → horizontal, column → vertical)
- **cross** = **perpendiculaire** à cette direction

```
// Row  : main = horizontal, cross = vertical
// Column : main = vertical, cross = horizontal
```

Exemple complet

```
import 'package:flutter/material.dart';

void main() => runApp(const MonApp());

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Row & Column')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,   //  tout aligné à
gauche
            children: [
              //  Titre
              const Text(
                'Ma liste',
                style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
              ),

              const SizedBox(height: 20),

              //  Une carte avec icône + texte + bouton
              Container(
                padding: const EdgeInsets.all(16),
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(12),
                  boxShadow: [
                    BoxShadow(
                      color: Colors.black.withOpacity(0.1),
                      blurRadius: 8,
                      offset: const Offset(0, 4),
                    ),
                  ],
                ),
                child: Row(
                  children: [
                    // Icône à gauche
                    const Icon(Icons.person, size: 40, color:
Colors.deepPurple),

                    const SizedBox(width: 16),

                    // Texte au milieu (prend l'espace restant)
                    Expanded(
                      child: Column(
```

```
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: const [
                          Text(
                            'Jean Dupont',
                            style: TextStyle(
                              fontSize: 18,
                              fontWeight: FontWeight.bold,
                            ),
                          ),
                          SizedBox(height: 4),
                          Text(
                            'jean@example.com',
                            style: TextStyle(color: Colors.grey),
                          ),
                        ],
                      ),
                    ),

                    // Bouton à droite
                    ElevatedButton(
                      onPressed: () {},
                      child: const Text('Voir'),
                    ),
                  ],
                ),
              ),

              const SizedBox(height: 20),

              //  Une ligne de statistiques
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: [
                  _Stat(col: Colors.blue, label: 'Posts', value: '42'),
                  _Stat(col: Colors.green, label: 'Amis', value: '128'),
                  _Stat(col: Colors.orange, label: 'Likes', value: '1.2k'),
                ],
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _Stat extends StatelessWidget {
  final Color col;
  final String label;
  final String value;
  const _Stat({required this.col, required this.label, required this.value});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(
          value,
          style: TextStyle(
            fontSize: 22,
            fontWeight: FontWeight.bold,
            color: col,
          ),
        ),
```

```
        Text(label, style: const TextStyle(color: Colors.grey)),
      ],
    );
  }
}
```

Padding et SizedBox

C'est quoi ?

Les deux servent à **créer de l'espace**, mais pas de la même façon :

- **`Padding`** = ajoute de l'espace **autour** d'un widget (comme une mousse autour d'un objet)
- **`SizedBox`** = un **rectangle invisible** de taille fixe (souvent utilisé comme "spacer" entre deux widgets)

Ce sont les **deux widgets les plus simples de Flutter**. Mais aussi les plus utilisés. Tu vas en mettre partout.

Différence visuelle

```
Padding :                    SizedBox :

┌── espace ──┐               [widget A]
│  ┌widget┐  │               [          ]  ← SizedBox (hauteur 20)
└──────────┘                 [widget B]

Enveloppe un widget          Est un widget à part entière
et ajoute de l'espace        (souvent un "trou" vide)
autour
```

PARTIE 1 — Padding

Étape 0 — Le minimum

```
Padding(
  padding: EdgeInsets.all(16),   //  OBLIGATOIRE
  child: Text('Salut'),
)
```

Effet : le texte "Salut" a **16 pixels d'espace** tout autour de lui.

> **`padding:` est obligatoire** dans un `Padding`. Sinon erreur.

Étape 1 — Les 4 variantes d'`EdgeInsets`

`EdgeInsets` = "espace sur les bords". Tu as 4 façons de le définir :

`EdgeInsets.all(x)` — même espace partout

```
padding: EdgeInsets.all(16)
// → 16 à gauche, 16 en haut, 16 à droite, 16 en bas
```

`EdgeInsets.symmetric(...)` — horizontal et vertical séparés

```
padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10)
// → 20 à gauche/droite, 10 en haut/bas
```

`EdgeInsets.only(...)` — côté par côté

```
padding: EdgeInsets.only(left: 10, top: 5, right: 20, bottom: 15)
// → chaque côté a sa valeur
// Les côtés non précisés valent 0
```

`EdgeInsets.fromLTRB(l, t, r, b)` — dans l'ordre

```
padding: EdgeInsets.fromLTRB(10, 5, 20, 15)
// → Left=10, Top=5, Right=20, Bottom=15
// Même chose que .only() mais plus court
```

Étape 2 — Padding dans une Row ou Column

Souvent, tu veux espacer **tous** les enfants d'une Row/Column. Deux méthodes :

Méthode A — envelopper **chaque** enfant :

```
Row(
  children: [
    Padding(padding: EdgeInsets.all(8), child: Text('A')),
    Padding(padding: EdgeInsets.all(8), child: Text('B')),
    Padding(padding: EdgeInsets.all(8), child: Text('C')),
  ],
)
```

Méthode B (mieux) — envelopper **toute la Row** :

```
Padding(
  padding: EdgeInsets.all(16),
  child: Row(
    children: [Text('A'), Text('B'), Text('C')],
  ),
)
```

La méthode B est plus propre quand tu veux le même espace global.

Étape 3 — Padding vs Container(padding:)

Les deux font la même chose :

```
// Avec Padding
Padding(
  padding: EdgeInsets.all(16),
  child: Text('Salut'),
)

// Avec Container
Container(
  padding: EdgeInsets.all(16),
  child: Text('Salut'),
)
```

Quand utiliser quoi ?

- `Padding` → quand tu veux **juste** de l'espace (plus léger, plus clair)
- `Container` → quand tu veux aussi une **couleur, bordure, taille**, etc.

Règle simple : si tu n'as besoin **que** de padding → utilise `Padding`. Sinon → `Container`.

Étape 4 — Le piège du Padding "inutile"

```
//  Inutile : Padding autour de rien
Padding(
  padding: EdgeInsets.all(16),
  child: Container(),       // Container vide = taille 0
)
```

Effet : rien. Le padding s'applique à un widget de taille 0 → toujours rien.

Solution : mets le padding **à l'intérieur** du Container, ou donne une taille à l'enfant.

PARTIE 2 — SizedBox

Étape 0 — Le minimum

```
SizedBox(
  width: 100,
  height: 50,
)
```

Effet : un rectangle **invisible** de 100×50. Tu ne vois rien, mais l'espace est occupé.

Un `SizedBox` **vide** sert à créer de l'espace entre deux widgets.

Étape 1 — Le "spacer" classique

C'est **l'utilisation n°1** de SizedBox : mettre de l'espace entre deux widgets dans une Row/Column.

```
Column(
  children: [
    Text('Titre'),
    SizedBox(height: 20),      //  20px d'espace vertical
    Text('Sous-titre'),
  ],
)
```

Effet :

```
Titre
← 20px →
Sous-titre
```

Dans une Row :

```
Row(
  children: [
    Icon(Icons.star),
    SizedBox(width: 8),        //  8px d'espace horizontal
    Text('Favori'),
  ],
)
```

Règle simple :

- Dans une **Column** → `SizedBox(height: ...)`
- Dans une **Row** → `SizedBox(width: ...)`

Étape 2 — Envelopper un widget pour lui donner une taille

SizedBox peut aussi **forcer la taille** d'un widget :

```
SizedBox(
  width: 200,
  height: 60,
  child: ElevatedButton(
    onPressed: () {},
    child: Text('Bouton large'),
  ),
)
```

Effet : le bouton fait exactement 200×60.

Utile pour :

- Forcer la taille d'un bouton
- Forcer la taille d'une image
- Donner une taille à un widget qui n'en a pas

Étape 3 — `SizedBox.shrink()` (taille zéro)

```
SizedBox.shrink()
```

Effet : un widget de **taille 0**. Ça ne prend aucune place.

Utile pour :

- Conditionner l'affichage d'un widget :

  ```
  condition ? Text('Oui') : SizedBox.shrink()
  ```

- Éviter les erreurs de "child is null"

Étape 4 — `SizedBox.expand()` (prend tout l'espace)

```
SizedBox.expand(
  child: Container(color: Colors.blue),
)
```

Effet : le SizedBox prend **tout l'espace disponible** et le Container bleu le remplit.

Utile pour : forcer un enfant à remplir son parent sans `Expanded`.

Étape 5 — SizedBox vs Container vide

Les deux peuvent créer de l'espace :

```
// Avec SizedBox
SizedBox(height: 20)

// Avec Container
Container(height: 20)
```

Différence :

- `SizedBox` → **léger**, uniquement pour la taille. **À préférer.**
- `Container` → lourd (beaucoup de paramètres), mais utile si tu veux une couleur etc.

Règle : pour un simple espace → utilise **toujours** `SizedBox`.

PARTIE 3 — Comparaison finale

Padding vs SizedBox vs Container

Besoin	Widget à utiliser
Espace **autour** d'un widget existant	`Padding`
Espace **entre** deux widgets (Row/Column)	`SizedBox`
Espace + couleur/bordure/déco	`Container(padding: ...)`
Forcer la **taille** d'un widget	`SizedBox(width, height, child: ...)`

Exemple comparatif

Objectif : mettre 20px entre "Titre" et "Sous-titre", et ajouter 16px d'espace autour de tout.

Avec Padding :

```
Padding(
  padding: EdgeInsets.all(16),
  child: Column(
    children: [
      Text('Titre'),
      SizedBox(height: 20),           // espace entre les deux
      Text('Sous-titre'),
    ],
  ),
)
```

Avec SizedBox partout :

```
Column(
  children: [
    SizedBox(height: 16),
    Text('Titre'),
    SizedBox(height: 20),
    Text('Sous-titre'),
    SizedBox(height: 16),
  ],
)
```

La **première** est plus claire. La deuxième est valide mais plus verbeuse.

Les pièges classiques

Piège 1 — Oublier `padding:` dans un `Padding`

```
//  ERREUR : padding is required
Padding(child: Text('Salut'))
```

Solution : toujours fournir un `EdgeInsets`.

Piège 2 — SizedBox sans width NI height

```
//  Ce SizedBox ne sert à rien
SizedBox()
```

Effet : taille 0, invisible, inutile.

Solution : donne-lui **au moins** une dimension.

Piège 3 — SizedBox dans une Column avec width mais pas height

```
//    Dans une Column, le width est ignoré (la Column impose la largeur)
Column(
  children: [
    SizedBox(width: 100),     //  inutile ici
    SizedBox(height: 20),     //  OK
  ],
)
```

Règle :

- Dans une **Column**, seul `height` compte.
- Dans une **Row**, seul `width` compte.

Piège 4 — Confondre `padding` et `margin`

Rappel du cours sur Container :

- `padding` = **espace intérieur** (dans la boîte)
- `margin` = **espace extérieur** (autour de la boîte)

```
Container(
  margin: EdgeInsets.all(20),     // espace AUTOUR
  padding: EdgeInsets.all(10),    // espace DEDANS
  color: Colors.blue,
  child: Text('Salut'),
)
```

Piège 5 — Mettre du Padding autour d'un Expanded

```
//  Expanded doit être DIRECTEMENT enfant d'une Row/Column
Row(
  children: [
    Padding(
      padding: EdgeInsets.all(8),
      child: Expanded(child: Container()),    //  erreur
    ),
  ],
)
```

Solution : inverse l'ordre :

```
Row(
  children: [
    Expanded(
      child: Padding(                          //  Padding DANS Expanded
        padding: EdgeInsets.all(8),
        child: Container(),
      ),
    ),
  ],
)
```

Exemple complet

```
import 'package:flutter/material.dart';

void main() => runApp(const MonApp());

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Padding & SizedBox')),
```

```
body: Padding(
  //  16px d'espace autour de TOUT le body
  padding: const EdgeInsets.all(16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      //  Titre
      const Text(
        'Ma carte',
        style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
      ),

      //  Espace vertical de 20px
      const SizedBox(height: 20),

      //  Une carte avec padding intérieur
      Container(
        // Padding INTÉRIEUR de 16px
        padding: const EdgeInsets.all(16),

        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(12),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(0.1),
              blurRadius: 8,
              offset: const Offset(0, 4),
            ),
          ],
        ),

        child: Row(
          children: [
            const Icon(Icons.star, color: Colors.orange, size: 32),

            //  Espace horizontal de 12px entre icône et texte
            const SizedBox(width: 12),

            //  Texte qui prend l'espace restant
            const Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    'Titre de la carte',
                    style: TextStyle(
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  // Petit espace vertical de 4px
                  SizedBox(height: 4),
                  Text(
                    'Description courte',
                    style: TextStyle(color: Colors.grey),
                  ),
                ],
              ),
            ),

            //  Bouton à droite
            ElevatedButton(
              onPressed: () {},
```

```
                    child: const Text('OK'),
                  ),
                ],
              ),
            ),

            //  Espace vertical de 24px
            const SizedBox(height: 24),

            // Padding symétrique (horizontal 8, vertical 4)
            Padding(
              padding: const EdgeInsets.symmetric(
                horizontal: 8,
                vertical: 4,
              ),
              child: Text(
                'Note : cette carte utilise Padding et SizedBox partout.',
                style: TextStyle(color: Colors.grey[600], fontSize: 12),
              ),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

Expanded et Flexible

C'est quoi ?

Les deux servent à **partager l'espace disponible** entre les enfants d'une `Row` ou d'une `Column`.

- **`Expanded`** = "je prends **TOUT** l'espace restant"
- **`Flexible`** = "je prends **ce dont j'ai besoin**, mais pas plus que l'espace dispo"

Ils **ne fonctionnent QUE** dans une `Row`, `Column` ou `Flex`. Ailleurs → erreur.

Image mentale

Imagine une **étagère horizontale** (Row) de 300px de large avec 3 objets :

Sans Expanded/Flexible :

```
[Objet A][Objet B][Objet C].......espace vide.......
```

Avec Expanded sur l'objet B :

```
[Objet A][——— Objet B remplit tout ———][Objet C]
```

Avec deux Expanded (flex 1 et flex 2) :

```
[Objet A][— B (1/3) —][——— C (2/3) ———]
```

PARTIE 1 — Expanded

Étape 0 — Le minimum

```
Row(
  children: [
    Text('Gauche'),
    Expanded(                          //  prend tout l'espace restant
      child: Container(color: Colors.blue),
    ),
    Text('Droite'),
  ],
)
```

Effet : "Gauche" et "Droite" gardent leur taille naturelle. Le Container bleu **remplit tout le milieu**.

`Expanded` doit être **directement** enfant d'une `Row`/`Column`. Pas d'intermédiaire.

Étape 1 — Comprendre le mécanisme

Quand tu mets `Expanded` :

1. Flutter calcule la taille des enfants **sans** `Expanded`
2. Il regarde **combien d'espace il reste**
3. Il donne **tout cet espace** aux enfants `Expanded` (partagé entre eux)

```
Row(
  children: [
    Icon(Icons.star),           // ~24px
    Expanded(child: Container()),  // prend TOUT le reste
    Icon(Icons.share),          // ~24px
  ],
)
```

Résultat : `[ ]` `[——————————]` `[ ]` sur toute la largeur.

Étape 2 — Partager en proportions (`flex`)

Si tu as **plusieurs** `Expanded`, tu peux contrôler la **proportion** avec `flex` :

```
Row(
  children: [
    Expanded(
      flex: 1,                                //  1 part sur 3
      child: Container(color: Colors.red),
    ),
    Expanded(
      flex: 2,                                //  2 parts sur 3
      child: Container(color: Colors.blue),
    ),
  ],
)
```

Effet :

```
[─── ROUGE (1/3) ───][──────── BLEU (2/3) ────────]
```

`flex` par défaut = 1. Donc `Expanded()` = `Expanded(flex: 1)`.

Exemples de proportions :

Configuration	Résultat
`Expanded() + Expanded()`	50% / 50%
`Expanded(flex: 1) + Expanded(flex: 2)`	33% / 66%
`Expanded(flex: 1) + Expanded(flex: 1) + Expanded(flex: 1)`	33% / 33% / 33%
`Expanded(flex: 2) + Expanded(flex: 3)`	40% / 60%

Étape 3 — Expanded pour centrer verticalement

Astuce classique : dans une `Column`, `Expanded` autour d'un enfant le fait **remplir la hauteur restante**.

```
Column(
  children: [
    AppBar-like widget en haut,
    Expanded(                               //  prend toute la hauteur restante
      child: Center(child: Text('Contenu centré')),
    ),
    BottomBar-like widget en bas,
  ],
)
```

Effet : le contenu du milieu est poussé au centre de l'espace entre le haut et le bas.

C'est le pattern le plus courant dans un `Scaffold.body` : une partie fixe en haut, une partie fixe en bas, et le milieu qui prend tout le reste.

PARTIE 2 — Flexible

Étape 0 — Le minimum

```
Row(
  children: [
    Flexible(                              //  prend ce qu'il peut, sans forcer
      child: Text('Un texte très long qui pourrait déborder de l\'écran...'),
    ),
    Icon(Icons.star),
  ],
)
```

Effet : le texte prend **juste ce dont il a besoin**, mais s'il est trop long il **se réduit** au lieu de déborder.

Étape 1 — Différence concrète avec Expanded

C'est **LE** point à comprendre.

Situation	`Expanded`	`Flexible`
L'enfant a besoin de **peu** d'espace	Il **prend tout** quand même	Il **prend juste** ce qu'il faut
L'enfant a besoin de **trop** d'espace	Il se **réduit**	Il se **réduit**
Comportement visuel	Force à remplir	Laisse respirer

Exemple visuel :

```
// Avec Expanded : le texte est "étiré" sur toute la largeur
Row(
  children: [
    Expanded(child: Text('Court')),        // prend toute la largeur
    Icon(Icons.star),
  ],
)

// Avec Flexible : le texte garde sa taille naturelle
Row(
  children: [
    Flexible(child: Text('Court')),        // garde sa taille, ne déborde pas
    Icon(Icons.star),
  ],
)
```

Règle pratique :

- Tu veux **remplir** → `Expanded`
- Tu veux **juste éviter le débordement** → `Flexible`

Étape 2 — Flexible avec `fit`

`Flexible` a un paramètre `fit` qui change son comportement :

```
Flexible(
  fit: FlexFit.tight,      //  se comporte comme Expanded
  child: Text('Salut'),
)

Flexible(
  fit: FlexFit.loose,      //  par défaut : prend ce qu'il veut
  child: Text('Salut'),
)
```

- `FlexFit.loose` (défaut) → prend **ce dont il a besoin**, jusqu'à l'espace dispo
- `FlexFit.tight` → prend **tout** l'espace dispo (identique à `Expanded`)

En fait : **`Expanded`** = **`Flexible(fit: FlexFit.tight)`**. C'est tout.

Étape 3 — Quand utiliser Flexible plutôt qu'Expanded

Cas typique : un `Text` à côté d'un widget fixe, et tu veux éviter le débordement.

```
Row(
  children: [
    Icon(Icons.person),
    SizedBox(width: 8),
    Flexible(                              //  le texte peut rétrécir si besoin
      child: Text('Un nom très très très long...'),
    ),
    Icon(Icons.arrow_forward),
  ],
)
```

Effet : si le nom est court, il prend sa taille. S'il est long, il se réduit et affiche "..." au lieu de déborder.

PARTIE 3 — Comparaison finale

Expanded vs Flexible

Aspect	`Expanded`	`Flexible`
Espace pris	**Tout** le restant	**Ce dont il a besoin**
Équivalent à	`Flexible(fit:`	`Flexible(fit:`

	`FlexFit.tight)`	`FlexFit.loose)`
Usage typique	Remplir un espace	Éviter le débordement
Peut être vide (Container)	Oui	Souvent taille 0

Expanded vs Flexible vs SizedBox

Besoin	Widget
Remplir tout l'espace restant	`Expanded`
Prendre ce qu'il faut, éviter le débordement	`Flexible`
Espace **fixe** entre deux widgets	`SizedBox`
Espace **proportionnel** entre widgets	`Expanded(flex: ...)`

Les pièges classiques

Piège 1 — `Expanded` en dehors d'une Row/Column

```
//  ERREUR : Expanded ne peut être qu'enfant de Row/Column/Flex
Container(
  child: Expanded(child: Text('Salut')),
)
```

Solution : `Expanded` **doit** être dans une `Row` ou `Column`.

Piège 2 — `Expanded` dans une Column scrollable

```
//  ERREUR : les contraintes de hauteur sont infinies
SingleChildScrollView(
  child: Column(
    children: [
      Expanded(child: Container()),      //  impossible
    ],
  ),
)
```

Pourquoi ? Dans un scroll, la hauteur est **infinie** → `Expanded` ne sait pas combien prendre.

Solution : enlève `Expanded`, ou utilise une hauteur fixe (`SizedBox`, `Container(height: ...)`).

Piège 3 — `Expanded` autour d'un `Padding` ou `Container`

```
//  Expanded doit être DIRECTEMENT dans Row/Column
Row(
  children: [
    Padding(
      padding: EdgeInsets.all(8),
      child: Expanded(child: Container()),   //  erreur
    ),
  ],
)

//  Bon ordre
Row(
  children: [
    Expanded(
      child: Padding(
        padding: EdgeInsets.all(8),
        child: Container(),
      ),
    ),
  ],
)
```

Règle : `Expanded` **tout en haut** de la chaîne, le reste à l'intérieur.

Piège 4 — `Expanded` sur un widget qui n'a pas besoin de remplir

```
//  Le bouton va s'étirer sur toute la largeur, c'est moche
Row(
  children: [
    Expanded(
      child: ElevatedButton(onPressed: () {}, child: Text('OK')),
    ),
  ],
)
```

Solution : si tu veux un bouton de taille naturelle, ne mets **pas** `Expanded`. Utilise plutôt `Row` avec `mainAxisAlignment`.

Piège 5 — Mélanger flex sans comprendre les proportions

```
Row(
  children: [
    Expanded(flex: 1, child: Container()),   // 1/4
    Expanded(flex: 1, child: Container()),   // 1/4
    Expanded(flex: 2, child: Container()),   // 2/4 = 1/2
  ],
)
```

Total des flex = 4. Chaque enfant prend `son flex / total`.

Exemple complet

```
import 'package:flutter/material.dart';

void main() => runApp(const MonApp());

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Expanded & Flexible')),
        body: Column(
          children: [
            //  Header fixe en haut
            Container(
              color: Colors.deepPurple,
              padding: const EdgeInsets.all(16),
              child: const Row(
                children: [
                  Icon(Icons.person, color: Colors.white, size: 40),
                  SizedBox(width: 12),
                  // Flexible : le nom peut rétrécir si trop long
                  Flexible(
                    child: Text(
                      'Jean Dupont',
                      style: TextStyle(
                        color: Colors.white,
                        fontSize: 20,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                  ),
                  Icon(Icons.settings, color: Colors.white),
                ],
              ),
            ),

            //  Zone du milieu : prend TOUT l'espace restant
            Expanded(
              child: Container(
                color: Colors.grey[200],
                child: const Center(
                  child: Text(
                    'Contenu principal\n(prend tout l\'espace restant)',
                    textAlign: TextAlign.center,
                    style: TextStyle(fontSize: 16),
                  ),
                ),
              ),
            ),
```

```
            //  Footer avec 3 zones proportionnelles
            Container(
              color: Colors.black87,
              padding: const EdgeInsets.all(12),
              child: Row(
                children: [
                  // 1/4 pour l'icône maison
                  Expanded(
                    flex: 1,
                    child: Icon(Icons.home, color: Colors.white, size: 28),
                  ),
                  // 2/4 pour le texte du milieu
                  Expanded(
                    flex: 2,
                    child: Text(
                      'Menu principal',
                      textAlign: TextAlign.center,
                      style: TextStyle(color: Colors.white70),
                    ),
                  ),
                  // 1/4 pour l'icône profil
                  Expanded(
                    flex: 1,
                    child: Icon(Icons.person, color: Colors.white, size: 28),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

`Text`

1. `Text` — afficher du texte

C'est le widget de base.

```
Text('Bonjour')
```

Tu peux le voir comme :

> **`Text` = je veux afficher une phrase à l'écran.**

Exemple :

```
Text('Bienvenue dans mon application')
```

Ajouter un style

```
Text(
  'Bienvenue',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
  ),
)
```

Ici :

```
Text
 │
 ├── texte → "Bienvenue"
 │
 └── style → TextStyle
                │
                ├── taille
                ├── couleur
                └── épaisseur
```

À retenir

```
Text('Bonjour')
```

→ afficher.

```
Text(
  'Bonjour',
  style: TextStyle(...),
)
```

→ afficher + personnaliser.

2. `TextStyle` — modifier l'apparence

`TextStyle` ne sert **pas à afficher** le texte.

Il sert à dire :

> **Comment mon texte doit-il être présenté ?**

Exemple :

```
TextStyle(
  fontSize: 20,
  color: Colors.red,
  fontWeight: FontWeight.bold,
  fontStyle: FontStyle.italic,
)
```

On l'utilise généralement dans `Text` :

```
Text(
  'Bonjour',
  style: TextStyle(
    fontSize: 20,
    color: Colors.red,
    fontWeight: FontWeight.bold,
  ),
)
```

Les propriétés importantes

Propriété	Rôle
`fontSize`	taille
`color`	couleur
`fontWeight`	épaisseur
`fontStyle`	normal / italique
`fontFamily`	police
`decoration`	souligné, barré...
`letterSpacing`	espace entre lettres
`wordSpacing`	espace entre mots
`height`	hauteur/interligne
`shadows`	ombre

Exemple simple

```
Text(
  'Flutter',
  style: TextStyle(
    fontSize: 30,
    fontWeight: FontWeight.bold,
    color: Colors.deepPurple,
  ),
)
```

Tu obtiens :

`FLUTTER`

en gros, gras et violet.

3. `TextAlign` — positionner le texte

`TextAlign` répond à la question :

> **Où placer mon texte horizontalement dans l'espace disponible ?**

Exemple :

```
Text(
  'Bonjour',
  textAlign: TextAlign.center,
)
```

Les principales valeurs :

```
TextAlign.left
TextAlign.right
TextAlign.center
TextAlign.justify
TextAlign.start
TextAlign.end
```

Exemple

```
Container(
  width: 300,
  child: Text(
    'Bonjour',
    textAlign: TextAlign.center,
  ),
)
```

Le `Container` fait 300 pixels de largeur.

Le texte est donc placé au milieu :



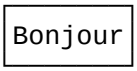
Pourquoi `TextAlign` ne semble parfois pas fonctionner ?

Regarde :

```
Row(
  children: [
    Text(
      'Bonjour',
      textAlign: TextAlign.center,
    ),
  ],
)
```

Le `Text` prend pratiquement seulement la place dont il a besoin.

Donc :

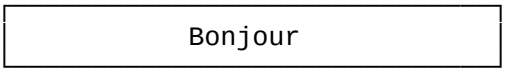


Il n'a pas d'espace supplémentaire dans lequel se centrer.

Solution

```
Row(
  children: [
    Expanded(
      child: Text(
        'Bonjour',
        textAlign: TextAlign.center,
      ),
    ),
  ],
)
```

Maintenant :



À retenir

> `TextAlign` fonctionne réellement lorsque `Text` dispose d'un espace plus grand que son contenu.

4. `TextOverflow` — gérer un texte trop long

Imagine une application qui affiche :

```
Le président directeur général de...
```

mais ton espace est très petit.

Tu dois dire à Flutter :

> Que dois-tu faire si le texte ne rentre pas ?

C'est exactement le rôle de `TextOverflow`.

`ellipsis`

Le plus utilisé :

```
Text(
  'Ceci est un très long texte qui ne rentre pas',
```

```
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
)
```

Résultat :

```
Ceci est un très long...
```

Les `...` indiquent que le texte a été coupé.

Les principales possibilités

`clip`

```
overflow: TextOverflow.clip
```

Coupe le texte.

```
Ceci est un texte...
```

mais sans forcément afficher `...`.

`ellipsis`

```
overflow: TextOverflow.ellipsis
```

Affiche :

```
Ceci est un texte...
```

C'est généralement le choix idéal pour les titres de listes.

`fade`

```
overflow: TextOverflow.fade
```

Le texte disparaît progressivement à la fin.

`visible`

```
overflow: TextOverflow.visible
```

Le texte peut dépasser de son espace.

5. `maxLines`

`TextOverflow` est souvent utilisé avec `maxLines`.

```
Text(
  'Un texte extrêmement long...',
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
)
```

On dit :

```
maxLines: 1
      ↓
une seule ligne autorisée

overflow: ellipsis
      ↓
si ça dépasse → ...
```

Exemple avec deux lignes

```
Text(
  'Voici un très long texte qui peut occuper deux lignes.',
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)
```

Résultat :

```
Voici un très long texte qui peut
occuper deux lignes...
```

6. `RichText` — plusieurs styles dans un même texte

Maintenant quelque chose d'important.

Avec :

```
Text(
  'Bonjour Jean',
)
```

tout le texte possède normalement le même style.

Mais imaginons que tu veuilles :

```
Bonjour Jean
```

```
        ↑
       bleu + gras
```

et :

```
Bonjour
```

reste normal.

C'est là que `RichText` / `Text.rich` intervient.

7. `TextSpan`

Pour comprendre `RichText`, il faut comprendre `TextSpan`.

Un `TextSpan` représente :

> **un morceau de texte.**

Exemple :

```
TextSpan(
  text: 'Bonjour',
)
```

Tu peux ensuite avoir plusieurs morceaux :

```
Bonjour | Jean | !
   ↓       ↓      ↓
normal    gras   normal
```

8. `Text.rich` — version pratique

Aujourd'hui, pour ce genre de besoin, je te conseille de retenir surtout :

```
Text.rich(...)
```

Exemple :

```
Text.rich(
  TextSpan(
    text: 'Bonjour ',
    children: [
      TextSpan(
        text: 'Jean',
        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Colors.blue,
        ),
```

```
      ),
      TextSpan(
        text: ' !',
      ),
    ],
  ),
)
```

Visuellement :

```
Bonjour Jean !
        ↑
   gras + bleu
```

9. Comment comprendre `Text.rich` ?

Regarde cette structure :

```
Text.rich(
  TextSpan(
    text: 'Bonjour ',
    children: [
      TextSpan(
        text: 'Jean',
        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Colors.blue,
        ),
      ),
    ],
  ),
)
```

On peut la lire comme ceci :

```
Text.rich
   │
   └── TextSpan
         │
         ├── "Bonjour "
         │
         └── TextSpan
               │
               ├── "Jean"
               └── style
                     ├── gras
                     └── bleu
```

10. `RichText` vs `Text.rich`

Tu verras les deux.

`RichText`

```
RichText(
  text: TextSpan(
    text: 'Bonjour',
  ),
)
```

`Text.rich`

```
Text.rich(
  TextSpan(
    text: 'Bonjour',
  ),
)
```

Pour débuter, retiens :

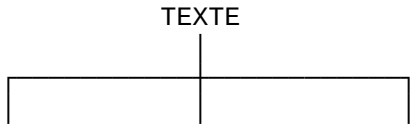
> **`Text.rich` est plus pratique parce qu'il fonctionne comme `Text` et accepte ses paramètres.**

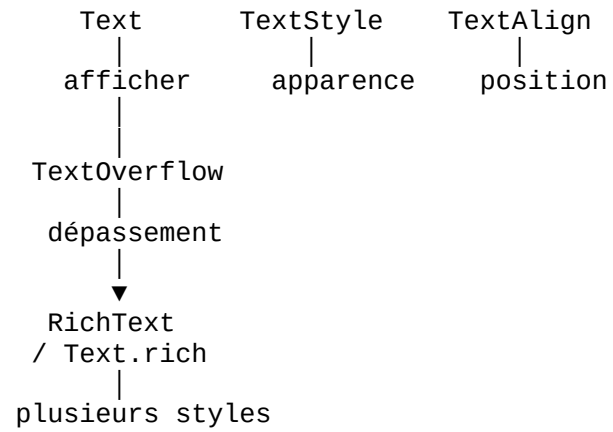
Par exemple :

```
Text.rich(
  TextSpan(
    text: 'Bonjour ',
    children: [
      TextSpan(
        text: 'Jean',
        style: TextStyle(
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
  textAlign: TextAlign.center,
)
```

11. Les cinq widgets/concepts à retenir

Maintenant, ne mémorise pas 50 propriétés. Mémorise cette logique :





12. Exemple complet

Voici un petit écran qui utilise les concepts essentiels :

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Widgets de texte'),
        ),

        body: Padding(
          padding: const EdgeInsets.all(20),

          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [

              // 1. Text simple
              const Text(
                'Bonjour Flutter',
              ),

              const SizedBox(height: 20),

              // 2. Text + TextStyle
              const Text(
                'Bienvenue dans mon application',
                style: TextStyle(
                  fontSize: 24,
                  fontWeight: FontWeight.bold,
```

```
                    color: Colors.blue,
                  ),
                ),

                const SizedBox(height: 20),

                // 3. TextAlign
                const SizedBox(
                  width: double.infinity,
                  child: Text(
                    'Ce texte est centré.',
                    textAlign: TextAlign.center,
                  ),
                ),

                const SizedBox(height: 20),

                // 4. TextOverflow
                const Text(
                  'Ceci est un texte très très long qui '
                  'ne doit pas dépasser la zone disponible.',
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                ),

                const SizedBox(height: 20),

                // 5. Text.rich
                Text.rich(
                  TextSpan(
                    text: 'Bonjour ',
                    children: [
                      TextSpan(
                        text: 'Jean',
                        style: TextStyle(
                          fontWeight: FontWeight.bold,
                          color: Colors.red,
                        ),
                      ),
                      TextSpan(
                        text: ', bienvenue !',
                      ),
                    ],
                  ),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

13. Comment choisir rapidement ?

Quand tu développes une application Flutter, pose-toi cette question :

Je veux simplement afficher du texte ?

```
Text('Bonjour')
```

Je veux modifier son apparence ?

```
Text(
  'Bonjour',
  style: TextStyle(...),
)
```

Je veux le centrer ?

```
Text(
  'Bonjour',
  textAlign: TextAlign.center,
)
```

Le texte est trop long ?

```
Text(
  '...',
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
)
```

Je veux plusieurs styles dans une phrase ?

```
Text.rich(
  TextSpan(
    ...
  ),
)
```

14. La fiche mémoire

Élément	Question à laquelle il répond	Exemple
`Text`	Qu'est-ce que j'affiche ?	`Text('Bonjour')`
`TextStyle`	Comment l'afficher ?	`fontSize`, `color`, `bold`
`TextAlign`	Où placer le texte ?	`center`
`TextOverflow`	Que faire s'il dépasse ?	`ellipsis`
`TextSpan`	Quel morceau de texte ?	`'Jean'`
`Text.rich`	Comment mélanger plusieurs styles ?	gras + rouge + normal

La formule à retenir

```
Text(
  'Mon texte',
  style: TextStyle(...),
  textAlign: TextAlign.center,
  maxLines: 1,
```

```
  overflow: TextOverflow.ellipsis,
)
```

Et lorsque tu veux plusieurs styles :

```
Text.rich(
  TextSpan(
    text: 'Bonjour ',
    children: [
      TextSpan(
        text: 'Jean',
        style: TextStyle(
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
)
```

Donc, pour l'instant, maîtrise surtout ces 5 idées :

`Text` = afficher
`TextStyle` = décorer
`TextAlign` = positionner
`TextOverflow` = gérer le dépassement
`Text.rich` = mélanger plusieurs styles.

Les boutons dans Flutter

L'idée est très simple :

Un bouton = l'utilisateur clique → l'application fait quelque chose.

Par exemple :

```
ElevatedButton(
  onPressed: () {
    print('Bouton cliqué');
  },
  child: const Text('Cliquer'),
)
```

Ici :

- `ElevatedButton` → le bouton

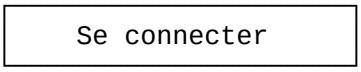
- `onPressed` → ce qui se passe quand on clique
- `child` → ce qu'on affiche dans le bouton

1. `ElevatedButton`

C'est le bouton **principal**, généralement utilisé pour une action importante.

```
ElevatedButton(
  onPressed: () {
    print('Connexion');
  },
  child: const Text('Se connecter'),
)
```

Visuellement :



Très important : `onPressed`

C'est lui qui donne **le comportement** au bouton.

```
onPressed: () {
  print('Bonjour');
},
```

On peut lire :

> Quand l'utilisateur appuie sur le bouton, exécute ce code.

2. `TextButton`

`TextButton` est un bouton **très discret**.

```
TextButton(
  onPressed: () {
    print('Mot de passe oublié');
  },
  child: const Text('Mot de passe oublié ?'),
)
```

On l'utilise souvent pour :

- "Annuler"
- "Voir plus"

- "Mot de passe oublié ?"
- "Créer un compte"

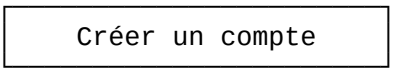
3. `OutlinedButton`

Il ressemble à un bouton avec une **bordure**.

```
OutlinedButton(
  onPressed: () {
    print('Inscription');
  },
  child: const Text('Créer un compte'),
)
```

Très pratique pour une **action secondaire**.

Par exemple :



4. `IconButton`

Ici, le bouton contient principalement une **icône**.

```
IconButton(
  onPressed: () {
    print('Suppression');
  },
  icon: const Icon(Icons.delete),
)
```

Par exemple :

```
IconButton(
  onPressed: () {
    print('Favori');
  },
  icon: const Icon(Icons.favorite),
)
```

Très utilisé dans :

- `AppBar`
- favoris
- supprimer

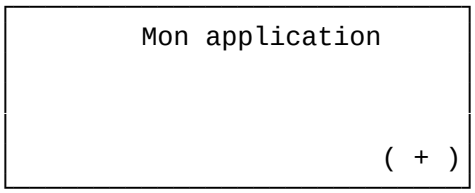
- modifier
- partager
- paramètres

5. `FloatingActionButton`

C'est le bouton rond qui **flotte généralement en bas de l'écran**.

```
FloatingActionButton(
  onPressed: () {
    print('Ajouter');
  },
  child: const Icon(Icons.add),
)
```

Exemple classique :



On l'utilise souvent pour l'action principale :

Ajouter un utilisateur
Ajouter une tâche
Ajouter un produit
Créer une note

6. La différence essentielle

Retenons ceci :

Widget	Utilisation
`ElevatedButton`	Action principale
`TextButton`	Action discrète
`OutlinedButton`	Action secondaire
`IconButton`	Action avec une icône
`FloatingActionButton`	Action principale flottante

7. Le `child`

Regarde :

```
ElevatedButton(
  onPressed: () {},
  child: const Text('Connexion'),
)
```

Le `child` est **ce qui se trouve à l'intérieur du bouton**.

Il peut être un texte :

```
child: const Text('Connexion')
```

mais également une icône :

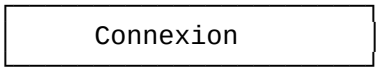
```
child: const Icon(Icons.login)
```

ou même plusieurs widgets.

Par exemple :

```
ElevatedButton(
  onPressed: () {},
  child: Row(
    mainAxisSize: MainAxisSize.min,
    children: const [
      Icon(Icons.login),
      SizedBox(width: 8),
      Text('Connexion'),
    ],
  ),
)
```

On obtient :



8. Désactiver un bouton

Très important.

Si :

```
onPressed: null
```

le bouton devient **désactivé**.

```
ElevatedButton(
  onPressed: null,
  child: const Text('Connexion'),
)
```

Alors l'utilisateur ne peut pas cliquer dessus.

À l'inverse :

```
ElevatedButton(
  onPressed: () {
    print('Connexion');
  },
  child: const Text('Connexion'),
)
```

le bouton est actif.

À retenir

`onPressed: null`

=>

bouton désactivé

`onPressed: () {}`

=>

bouton actif

9. Personnaliser un bouton

On peut utiliser `style`.

```
ElevatedButton(
  onPressed: () {},
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.deepPurple,
    foregroundColor: Colors.white,
  ),
  child: const Text('Connexion'),
)
```

Ici :

`backgroundColor`

→ couleur du fond.

`foregroundColor`

→ couleur du texte et généralement des icônes.

10. Modifier la taille

On peut utiliser :

`minimumSize`

Exemple :

```
ElevatedButton(
  onPressed: () {},
  style: ElevatedButton.styleFrom(
    minimumSize: const Size(200, 50),
  ),
  child: const Text('Connexion'),
)
```

On demande donc :

> Je veux au minimum un bouton de 200 × 50.

11. Arrondir le bouton

```
ElevatedButton(
  onPressed: () {},
  style: ElevatedButton.styleFrom(
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(20),
    ),
  ),
  child: const Text('Connexion'),
)
```

Plus `circular()` est grand, plus les coins sont arrondis.

12. Exemple concret : écran de connexion

Maintenant on met tout ensemble.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
```

```
const MonApp({super.key});

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      appBar: AppBar(
        title: const Text('Connexion'),
      ),

      body: Padding(
        padding: const EdgeInsets.all(20),

        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [

            const Text(
              'Bienvenue',
              style: TextStyle(
                fontSize: 28,
                fontWeight: FontWeight.bold,
              ),
            ),

            const SizedBox(height: 30),

            ElevatedButton(
              onPressed: () {
                print('Connexion');
              },
              style: ElevatedButton.styleFrom(
                minimumSize: const Size(
                  double.infinity,
                  50,
                ),
              ),
              child: const Text('Se connecter'),
            ),

            const SizedBox(height: 10),

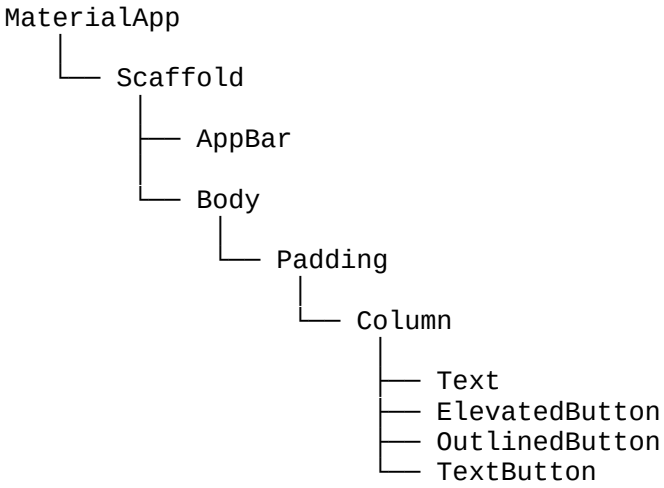
            OutlinedButton(
              onPressed: () {
                print('Inscription');
              },
              style: OutlinedButton.styleFrom(
                minimumSize: const Size(
                  double.infinity,
                  50,
                ),
              ),
              child: const Text('Créer un compte'),
            ),

            TextButton(
              onPressed: () {
                print('Mot de passe oublié');
              },
              child: const Text(
                'Mot de passe oublié ?',
              ),
            ),
```

```
            ],
          ),
        ),
      ),
    );
  }
}
```

Ce que tu dois comprendre

L'arbre est essentiellement :



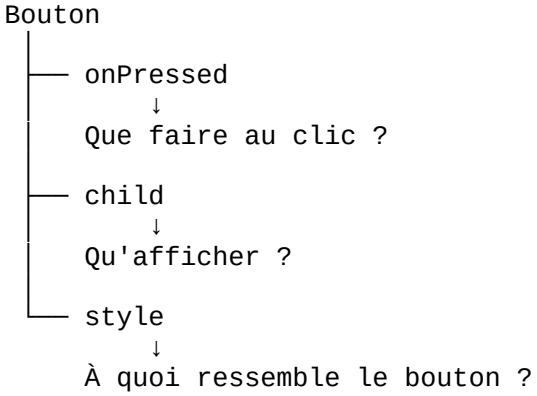
C'est exactement cette logique d'**arbre de widgets** qu'il faut commencer à maîtriser.

13. Une règle très importante

Ne retiens pas simplement :

> "`ElevatedButton` sert à faire un bouton."

Comprends plutôt :



Donc :

```
ElevatedButton(
```

```
  onPressed: () {
    // ACTION
  },

  style: ...,

  child: const Text(
    'Connexion',
  ),
)
```

est la structure fondamentale.